A MASTERCLASS

Vibecoding.

Direct, verify, ship — with Claude Code.

> a four-hour masterclass on agentic engineering

ELIAS JELINEK

Place Beyond Bytes · Chair of Sustainability and Innovation in Digital Ecosystems

University of Duisburg-Essen · Summer 2026

WHO IS RUNNING THIS

Hi. I'm Elias.

WHAT I DO

I research AI agents for the founding phase of startups - how teams can use agentic systems to go from idea to MVP to first revenue with a fraction of the headcount.

On the side, I build the thing I research:

QUERSHIFT

Agentic AI startup

Building exactly the kind of system this masterclass teaches.

DIFFUSIONE

Side ventures

Where I keep building things by hand.

Plus a chair seat at the University of Duisburg-Essen - same topic.

An hour of theory and demos. Then three hours of doing.

Agenda.

01	What's possible	Six live demos — code, multi-agent, browser, ML, automation
02	The mindset shift	From vibe-coding to agentic engineering
03	Coding workflows — the core	Specs, plans, repo setup, tests, orchestrator, deploy
04	Beyond coding	Daily automations, project management, Second Brain
05	AI engineering	Using Claude Code to scaffold ML training, RAG, evals
06	The wider toolbox	Cursor, Cline, Aider, Codex — when to use what
07	Your turn	Three hours of hands-on, in pairs, in this repo

By 18:00 today,

you'll have shipped something real.

Pick one: a CLI tool - a data script - a small web scraper - a trained model.

First, two terms. They sound similar. They're not.

Vibe Coding.

> *“I described what I wanted in plain English and the AI built it.”*

Fast. Magical. Often broken on the second input.

Great for prototypes. Great for solo weekends. Less great for clients.

The vibe is the prompt. The result is whatever Claude felt like producing.

Where this masterclass actually lives.

Agentic Engineering.

VIBE CODING

Prompt -> result.

You ask. AI delivers. You verify (or don't).

Works for: throwaway scripts, quick prototypes, fun.

AGENTIC ENGINEERING

Spec -> plan -> verify -> repeat.

You stay the engineer. AI does the typing. Every step has evidence.

Works for: clients, production systems, your career.

Today is about the right side. The left is just where we start.

Forty seconds. Enough to understand what's actually happening.

What is an LLM?

01

TRAINED ON TEXT

Read most of the internet, plus books, code, papers - once.

02

PREDICTS NEXT TOKEN

Given a sequence so far, returns the most likely next chunk.

03

REPEATED

Run that prediction over and over. The result looks like reasoning.

It is not a database. It is not a calculator. It is a very good guesser.

Everything we'll learn today is about how to make the guessing useful.

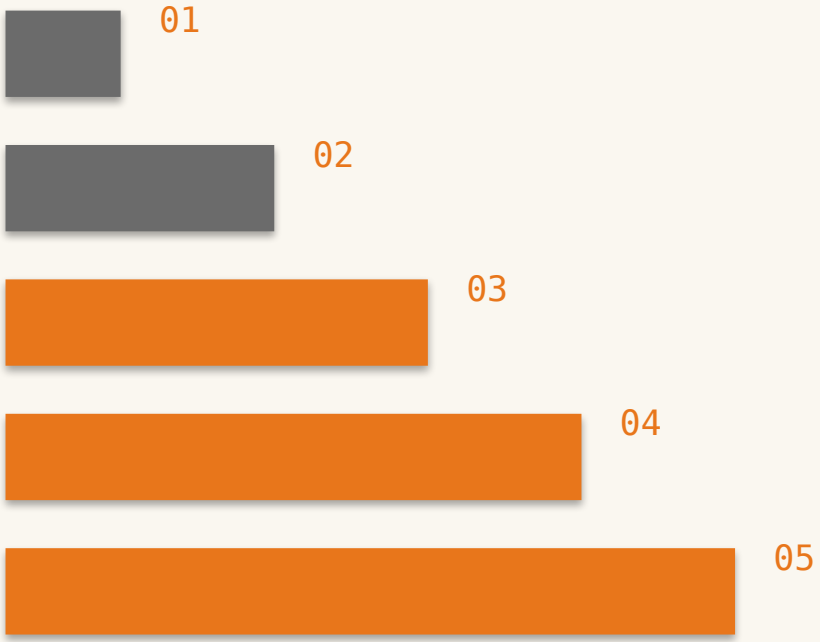
Same underlying tech. Very different products.

Claude - and where it sits.

MODEL	VENDOR	WHAT IT'S BEST AT
CHATGPT	OpenAI	General-purpose chatbot, biggest mindshare, broadest API
CLAUDE	Anthropic	Strong on long context, careful reasoning, agentic workflows
GEMINI	Google	Tight Workspace integration, strong multi-modal
LLAMA / MISTRAL	Open weights	Run locally or on your own infra; smaller, hackable
CLAUDE CODE	Anthropic CLI	What we'll use today - a terminal-native agent harness

Where AI lives in your workflow. Each rung needs different tools.

Five levels of AI integration.



CHATBOT
ChatGPT, Claude.ai web

Tab open. Copy-paste. Manual.

IDE INTEGRATION
Cursor, Copilot, Continue

AI in your editor. Inline completions, side-chat.

AGENTIC CODING
Claude Code, Aider, Codex CLI

Terminal-native. Plans, edits, runs tests.

MULTI-AGENT
Agent Teams, custom harnesses

Orchestrator + subagents. Parallel work, verified handoffs.

AUTO-DEV
Anthropic Routines, Devin-class

Continuous: spec in, deployed code out. Human approves milestones.

Today we operate at levels 3 and 4 - and look at level 5.

What turns a chatbot into an agent. What Claude Code actually is.

What is an agent harness?

THE SHORT VERSION

A program that gives an LLM the ability to do things.

Read files. Run commands. Call APIs. Loop until done. Stop when stuck. Ask for approval.

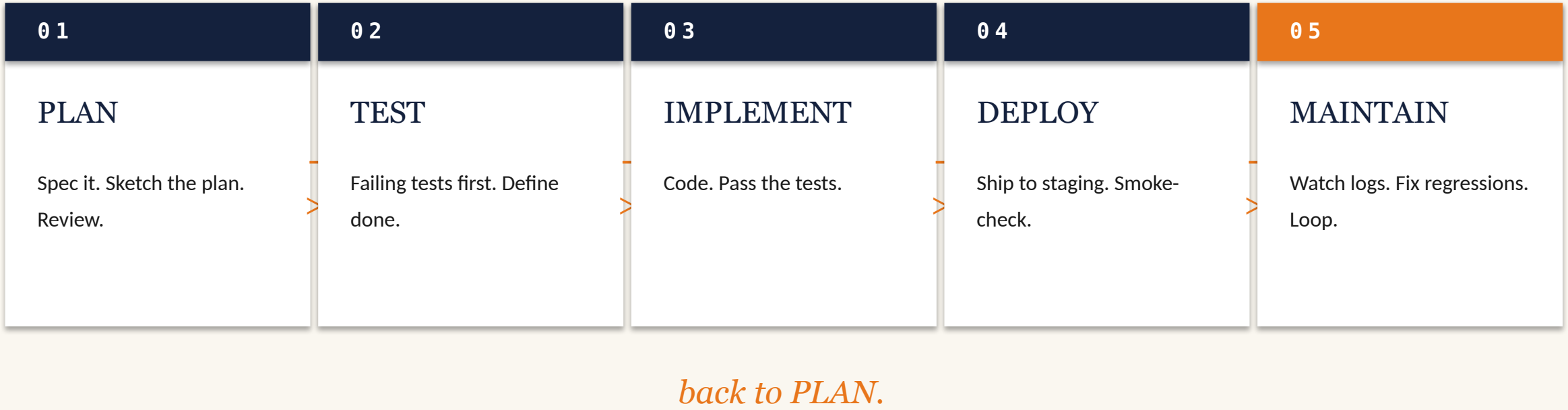
The LLM is the brain. The harness is the body.

WHAT'S INSIDE

LLM CORE	The model. Predicts the next action.
TOOL LAYER	Read, Write, Edit, Bash, WebFetch, MCP servers.
CONTEXT MANAGER	What's in the prompt right now. Files, memory, history.
LOOP / PLANNER	Decides what to do next. Detects done. Detects stuck.
PERMISSIONS	Asks before destructive actions. Sandboxes.

Real software has more than five steps. Here's the bigger loop.

Plan -> Test -> Implement -> Deploy -> Maintain.



back to PLAN.

Most teams stop at step 3. The whole point of agentic work is step 5.

DISCUSSION · 04 MINUTES

What's the most boring, most repetitive thing you do every week?

HOW TO DISCUSS

Two minutes with your buddy. Then two pairs share with the room.

From one paragraph to a working app, in under five minutes.

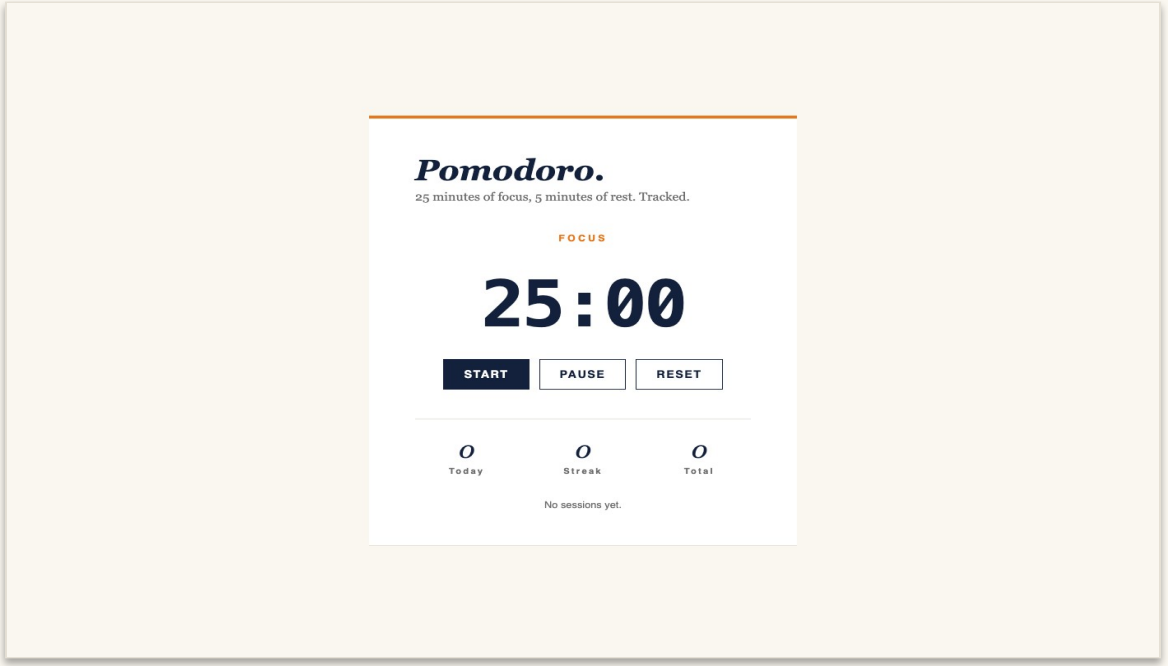
Vibe-build an MVP.

```
> claude

"Build a Pomodoro timer.
 25 min focus, 5 min rest.
 Stats persist. Editorial style.
 Cream + coral. No frameworks."
```

ONE PROMPT

No framework. No build step. Spec-tight prompt + plan mode + render = working app.



Final UI · localStorage persistence · zero dependencies

But this is the easy part. Now what about real software?

How production-ish features actually move through Claude Code.

Spec → Plan → Implementation.

SPEC

demos/02-…/spec.md

```
# Spec — CSV Export

Owner: Elias
Status: approved

## Checkable criteria
1. export_to_csv(rows, out) ...
2. Header row present.
3. UTF-8, RFC 4180.
4. Empty rows → header only.
5. Refuses to overwrite.

## Out of scope
- xlsx output
- multi-currency
```

PLAN

demos/02-…/plan.md

```
# Plan — CSV Export

→ parser.py: add export_to_csv()
→ tests/test_export.py: 3 tests

## Assumptions
- csv.DictWriter, comma delim
- Path-wrap out_path

## Done when
- 3 tests green
- empty.csv has 1 line
```

DIFF + TEST

parser.py + pytest

```
+ CSV_COLUMNS = ('date',
    'shop', 'item',
    'price', 'currency')

+ def export_to_csv(rows, out):
+     if out.exists():
+         raise FileExistsError

test_round_trip       PASSED
test_empty_header     PASSED
test_refuses_over     PASSED

=== 3 passed in 0.04s ===
```

Three artifacts. Each reviewable. Each persists after the code ships.

One conversation. Three parallel investigators. Synthesized in seconds.

Multi-agent in motion.

```
> claude

You: Spawn three Explore subagents in parallel on this codebase. Ask:
     (1) where is URL validation done?
     (2) external dependencies and pinned versions?
     (3) any password or token leaking into logs?
     Synthesize.

[orchestrator] Dispatching 3 Explore agents in parallel...

  AGENT 1  →  validators ⏎
  AGENT 2  →  dependencies ⏎
  AGENT 3  →  security audit ⏎

[2.4s] AGENT 1 returned (1 finding)
[2.7s] AGENT 2 returned (4 findings)
[3.1s] AGENT 3 returned (2 findings, HIGH severity)
Synthesis:
  URL validation lives in validators.py:9. The shortener has a weaker
```

Three parallel agents. One synthesis. You read it like a tech lead.

Claude Code drives a real browser — extraction, automation, admin.

Browser-driving Claude.

```
> claude

You: Open arxiv.org/list/cs.AI/recent in Chrome. Extract the top 5 paper
     titles with their submission dates. Output as JSON.

[claude] Using Chrome MCP...

  ✓ tab_create  https://arxiv.org/list/cs.AI/recent
  ✓ get_page_text                                  (1.2s)
  ✓ extracted 5 entries

[
  {
    "title": "Direct Preference Optimization for Code Agents",
    "authors": "Wang, Chen, Müller",
    "submitted": "2026-05-13",
    "id": "2605.04812"
  {
    "title": "Spec-Driven Generation: A Survey of Production Workflows",
    "authors": "Park, Iyer, Schmidt",
```

Same model, different surface. Code is just one of the surfaces.

20-newsgroups, four topics. Claude scaffolds the pipeline; you read the numbers.

Claude trains an ML model.

```
Loading 20-newsgroups subset (4 topics)...
Train: 2323 docs  ·  Test: 1546 docs
Classes: graphics, hockey, space, guns

Training...

Test accuracy:  0.895

Confusion matrix (rows = actual, columns = predicted):
           graphics    hockey     space      guns
  graphics       353         3        28         5
    hockey         4       369        15        11
     space        23         7       346        18
      guns        13         2        34       315
```

89.5% accuracy across four topics. Real dataset. Real numbers.

Claude Code writes to your vault. Every session ends as a note.

Notes that write themselves.

```
~/Documents/Obsidian Vault/
├── 00-Inbox/
│   └── Inbox.md
├── 01-Projects/
│   ├── Vibecoding Masterclass.md
│   ├── Inexogy Invoice Service.md
│   └── ECIS 2026 Paper.md
├── 03-Knowledge/
│   ├── Spec-Driven Development.md
│   ├── Multi-Agent Patterns.md
│   └── Claude Code Hooks.md
├── 04-People/
│   ├── Mahnoor Shah.md
│   └── Khan, Güzey, Nielsen.md
├── 05-Meetings/
│   └── 2026-05-14 SUST Triannual.md
├── 08-Sessions/
│   ├── 2026-05-14 Inbox.md      ← AUTO-GENERATED
│   └── 2026-05-13 Forderung.md  ← AUTO-GENERATED
```

HOW IT WORKS

A SessionEnd hook fires when you close Claude Code.

The hook writes a markdown note to your Obsidian vault — what you worked on, decisions made, open questions.

Tomorrow's session reads it back. Nothing gets lost.

Your brain has limits. Your filesystem doesn't.

Six demos. One shape.

The thread.

01 You wrote the spec, not the code.

The prompt was a tight description of what you wanted.

02 Claude proposed a plan before acting.

Even on small demos, there's a plan you could intercept.

03 The output was reviewable.

Test green, screenshot, diff, accuracy — evidence each time.

04 You stayed the human in the loop.

Claude did the typing. You did the judging.

Now let's unpack how to actually run this discipline.

Now let's unpack how this actually works.

The mindset → the workflow → the wider toolbox.

Vibecoding → Agentic Engineering.

A mindset shift, not a tool swap.

What the marketing says.

The promise.

> *"Describe what you want in natural language — the AI writes the code for you."*
>
> — *every AI coding demo on every conference stage, 2024-2026*

And to be fair — sometimes it really does work like that.

What you ship if you stop there.

And the catch.

✗ Hallucinated APIs	calls a function that doesn't exist
✗ Silent scope creep	asked for one feature, got four
✗ Plausible bugs	looks right; fails on the second input
✗ False “done”	tests not run, output not seen
✗ No paper trail	no spec, no plan, can't reproduce decisions

We need engineering discipline back. With the AI, not despite it.

0

Three lies AI coding tools tell us.

LIE 1

“It works.”

It looks like it works. The imports look right, the signature looks right, the README looks right.

But did anyone run it?

DISCIPLINE

Verification before done.

LIE 2

“I understand.”

It pattern-matched your keywords.

It may have grasped the surface. It rarely grasps the intent.

DISCIPLINE

Write a spec. Or make Claude write one — then review it.

LIE 3

“I'm done.”

“Done” is a status, not a feeling. Done means the code is written, the tests pass, the output was seen — by you.

DISCIPLINE

Stop conditions. Show your work.

The five steps. In this order. Every time.

The rhythm.

01

Spec

What you want.

Checkable success criteria.

02

Plan

Recipe between spec and code. Reviewable.

03

Test

At least one failing test before code.

04

Implement

The boring step.

05

Verify

Run it. See the output.

Show evidence.

Skip any one of these and you didn't really ship.

PART 03

→

Coding workflows.

Where the actual discipline lives.

A spec is not a wish list. It's a contract you can check.

What is a spec?

VAGUE INTENT	CHECKABLE SUCCESS CRITERIA
“Make the dashboard faster.”	Dashboard renders < 2s at p95.
“Improve the user experience.”	All forms submit < 250ms.
“Handle errors properly.”	Invalid input → 400 + body.error.
“Make it work for our enterprise client.”	Multi-tenant: row isolation per tenant_id.

If you can't tell whether you're done, you don't have a spec.

Five elements. Always.

Anatomy of a good spec.

01	Why	The motivation. Who asked, what problem it solves.
02	What	The thing being built — one paragraph, no jargon.
03	Checkable success criteria	Numbered list. Each item testable. No interpretation.
04	What it must NOT do	Failure modes, scope boundaries, things to refuse.
05	Out of scope	Explicit exclusions. Prevents scope creep mid-build.

`See demos/02-spec-plan-impl/spec.md for a real example.`

Clients give you wishes. You convert wishes into checkable criteria.

Capturing specs from clients.

› *“What happens when this works?”*	Forces them to describe an outcome, not a feature.
› *“What does failure look like?”*	Surfaces the unspoken edge cases.
› *“Who else needs to sign off?”*	Reveals the actual decision tree, not the org chart.
› *“What's explicitly OUT of scope?”*	The hardest question. The most valuable answer.
› *“Can I read the spec back to you?”*	The demo-it-back loop. Catches misalignment before code.

Code rots fastest. Specs rot slowest. Treat them accordingly.

Specs outlive the code.

Code is an implementation.

It changes weekly.

The spec is the contract.

It changes when the deal changes.

WHY THIS MATTERS

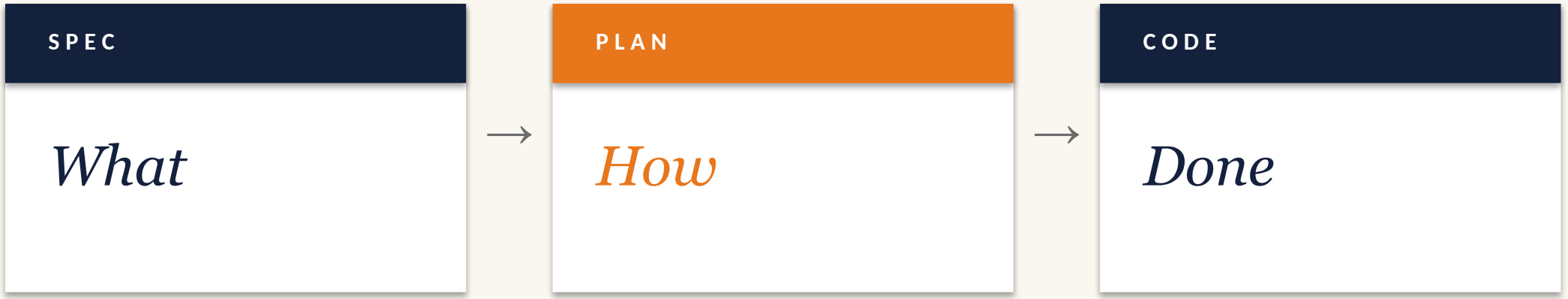
When a regression hits production six months later, you don't ask “what did the code do.” You ask “what was this supposed to do.”

The spec answers that. The code does not.

Store specs in version control. Reference them in PRs. They are the contract.

The recipe between the spec and the code.

What is a plan?



Without a plan, the spec stays an idea and the code becomes a guess.

A good plan is shorter than you expect.

Five questions on every plan Claude proposes. Always.

Anatomy of a good plan.

✓ States its assumptions out loud	*“I'm assuming X.” Not silent guessing.*
✓ Names the exact files it will touch	*parser.py:42, not “the parser”.*
✓ Proposes at least one test	*“Here's the test that proves this works.”*
✓ Stays scoped to what you asked	*Count the verbs. Cut bonus features.*
✓ Doesn't claim what it can't know	*“Following the existing convention” — which?*

Plan mode = read-only exploration before any approval-gated change.

Plan mode in Claude Code.

```
> claude /plan

Task: refactor the auth middleware.

[exploring codebase ...]
[reading 14 files in ./src/auth/]
[reading 3 test files]

Plan
1. Extract token validation into auth/tokens.py
2. Replace direct DB calls with the new repo class
3. Add 5 unit tests in tests/auth/test_tokens.py

Assumptions
- Keeping the existing JWT_SECRET env var
- NOT changing the public API surface

Approve? (y/n/edit)
```

Nothing changes on disk until you approve. That's the point.

From the plan above to the change on disk. Side by side.

Plan -> approved -> outcome.

THE APPROVED PLAN

```
Plan
1. Extract token validation
   -> auth/tokens.py
2. Replace direct DB calls
3. Add 5 unit tests

Approved by Elias at 14:23
```

THE OUTCOME ON DISK

```
$ git diff --stat
  auth/middleware.py  | -28 +6
  auth/tokens.py      | +42 (new)
  tests/test_tokens.py| +73 (new)
  3 files, +121 -28

$ pytest auth/
  5 passed in 0.06s

$ git status
  clean working tree
```

The plan was the spec. The outcome was the implementation. Both reviewable.

A layout Claude Code can navigate without guessing.

Repo setup for agent-friendly work.

```
project/
├── README.md          ← what + how to run
├── CLAUDE.md          ← conventions, ≤ 200 lines
├── specs/             ← versioned specs
│   └── 0001-csv-export.md
├── src/
│   └── parser.py
├── tests/
│   └── test_parser.py
├── .claude/
│   ├── hooks/         ← deterministic automation
│   │   └── pre-commit.sh
│   └── skills/        ← capability uplift
│       └── csv-export.md
└── .github/
    └── workflows/     ← CI as a verify gate
```

PRINCIPLES

- One CLAUDE.md per repo, root level
- Specs as first-class artifacts
- Hooks for the things you forget
- Skills for the things you repeat
- CI = automated verify gate
- No deep nesting

Read at session start. Sticks across every prompt. Keep it short.

CLAUDE.md — project DNA.

```
# Project conventions

## The rhythm
SPEC → PLAN → TEST → IMPL → VERIFY

## When you're working here
- Use plan mode for >1 file changes
- Tests before implementation
- Never claim done without output
- Cite file:line for every reference
- No real client data, ever

## What to avoid
- Long try/except that swallow errors
- Premature abstraction (< 3 callers)
- Comments that say what code does
```

RULES

- ≤ 200 lines. Hard cap.
- Conventions, not code.
- “Why” over “what”.
- Update when the project drifts.
- Run `claude /init` to bootstrap.
- It's advisory — ~80% compliance.

When 80% isn't good enough. Deterministic, every time.

Hooks — automate the must-haves.

```
# .claude/settings.json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "command": "ruff format --check"
    }],
    "PostToolUse": [{
      "matcher": "Edit",
      "command": "pytest -q tests/"
    }],
    "SessionEnd": [{
      "command": "~/.claude/session-note.sh"
    }]
  }
}
```

USE FOR

- Linting and formatting
- Security checks before commit
- Auto-tests on file changes
- Session notes to your vault
- Logging, telemetry
- Refusing forbidden patterns

The hook fires. The wrong commit gets blocked. The right one ships.

Hooks in action.

FAILING COMMIT · BLOCKED

```
$ git commit -m "add CSV export"

[hook] pre-commit: ruff format --check
Would reformat: parser.py
[hook] pre-commit: pytest -q
  test_round_trip       FAILED
  AssertionError at line 47

Commit blocked.
$
```

PASSING COMMIT · SHIPS

```
$ git commit -m "add CSV export"

[hook] pre-commit: ruff format --check
All formatted.
[hook] pre-commit: pytest -q
  3 passed in 0.04s

[main 7a3f2c1] add CSV export
 2 files changed, 49 +
$
```

The verify step is now part of the commit. You can't forget it.

Two kinds: give Claude new abilities, or encode your taste.

Skills — capabilities + preferences.

CAPABILITY UPLIFT

Give Claude a new ability.

- `pdf` — read and edit PDF files
- `browser` — open pages, fill forms
- `playwright` — generate E2E tests
- `fastapi-backend-design`
- `database-design`
- `ml-training-pipelines`

ENCODED PREFERENCE

Guide existing capabilities.

- `german-client-communication`
- `gdpr-dsgvo-compliance`
- `meeting-to-actions`
- `writing-plans (this discipline!)`
- `polished-ui-design`
- `verification-before-completion`

Skills are markdown. MCP is JSON-RPC. Skills know things; MCP does things.

Claude can now act outside the terminal — through any MCP-speaking tool.

MCP — the action layer.

```
$ claude mcp add notion --token=...
$ claude mcp add slack --workspace=...
$ claude mcp add postgres --conn=...
$ claude mcp add chrome

$ claude
> "Read today's standup notes from Slack,
   create a Notion page summarising blockers,
   then open chrome to file the bug."

[claude] mcp__slack__read_channel ...
[claude] mcp__notion__create-pages ...
[claude] mcp__chrome__navigate ...
[claude] mcp__chrome__form_input ...
```

POPULAR MCP SERVERS

- Notion (docs, databases)
- Slack (channels, threads)
- Linear (issues, projects)
- Gmail / Google Calendar
- Chrome (browser automation)
- GitHub (via gh CLI)
- Postgres / SQLite / DuckDB
- Filesystem with permissions

Tests aren't safety nets. They're the spec, executable.

Unit tests — what they actually are.

```python
# tests/test_export.py
from parser import export_to_csv, parse_receipts

def test_round_trip(tmp_path):
    rows = parse_receipts(SAMPLE)
    out = tmp_path / "out.csv"
    export_to_csv(rows, out)
    assert out.read_text().count("\n") == 14

def test_empty_writes_header_only(tmp_path):
    out = tmp_path / "empty.csv"
    export_to_csv([], out)
    assert out.read_text().splitlines() == [
        "date,shop,item,price,currency"]

def test_refuses_overwrite(tmp_path):
    out = tmp_path / "x.csv"; out.touch()
    with pytest.raises(FileExistsError):
        export_to_csv([], out)
```

GOOD UNIT TESTS

- Test one behavior
- Read like a spec
- Fail loudly with a clear message
- Don't share state
- Run in milliseconds
- Cover happy path AND one failure

Standard TDD. Just with Claude as the typist.

Red, green, refactor.

RED	GREEN	REFACTOR
Failing tests first.	Simplest code that passes.	Only if simpler. Else stop.

*Claude *will* try to write the function first. Correct it. Always.*

Web UI? Then Playwright tests, autogenerated from the spec.

Playwright + Claude Code.

```
// Generated by Claude Code from spec.md in 90 seconds.
import { test, expect } from "@playwright/test";

test.describe("Pomodoro", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("file://" + process.cwd() + "/demos/01-mvp-pomodoro/index.html");
  });

  test("starts at 25:00 in FOCUS mode", async ({ page }) => {
    await expect(page.locator("#timer")).toHaveText("25:00");
    await expect(page.locator("#mode")).toHaveText("FOCUS");
  });

  test("ticks down after START", async ({ page }) => {
    await page.click("#start");
    await page.waitForTimeout(1100);
    const txt = await page.locator("#timer").textContent();
    expect(txt).toBe("24:59");
  });
```

Generated from the spec for the Pomodoro app. 90 seconds.

Green tests are necessary. Not sufficient.

Verify properly.

01	Run the feature end-to-end	Not the test. The actual feature, as a user would.
02	Inspect the artifact	Open the CSV. Read the rendered page. Curl the endpoint.
03	Try one adversarial input	Empty list. Special chars. Concurrency. The thing that scared you.
04	Compare to the spec	Each checkable criterion: tick or not?
05	Show evidence	Paste output, attach screenshot, save the log. Make it visible.

Feature by feature. Evidence each loop. No skipping.

Iteration cycles.

```
loop:
    1. pick the next spec criterion
    2. /plan — propose, review, approve
    3. write failing test
    4. implement
    5. run test → green
    6. run feature end-to-end → works
    7. paste evidence
    8. check against spec — drift? regenerate plan
    9. commit with spec reference in message
   10. ↺

# do not move on while step 6 or 7 is incomplete.
```

Small loops. Small reviews. Small risks. The compounding wins.

The silent project killer in agentic workflows.

What is spec drift?

- Plan grows beyond the spec — Claude offers a “bonus” feature, you accept.
- Spec ambiguity resolved in conversation — but not written back.
- Tests cover the new shape, not the original criteria — green still ships drift.
- Stakeholder asks for “one tweak” mid-build — not added to the spec.
- Subagent decides on its own what done means — you don't notice.

By month two, no one can tell you what the system is supposed to do.

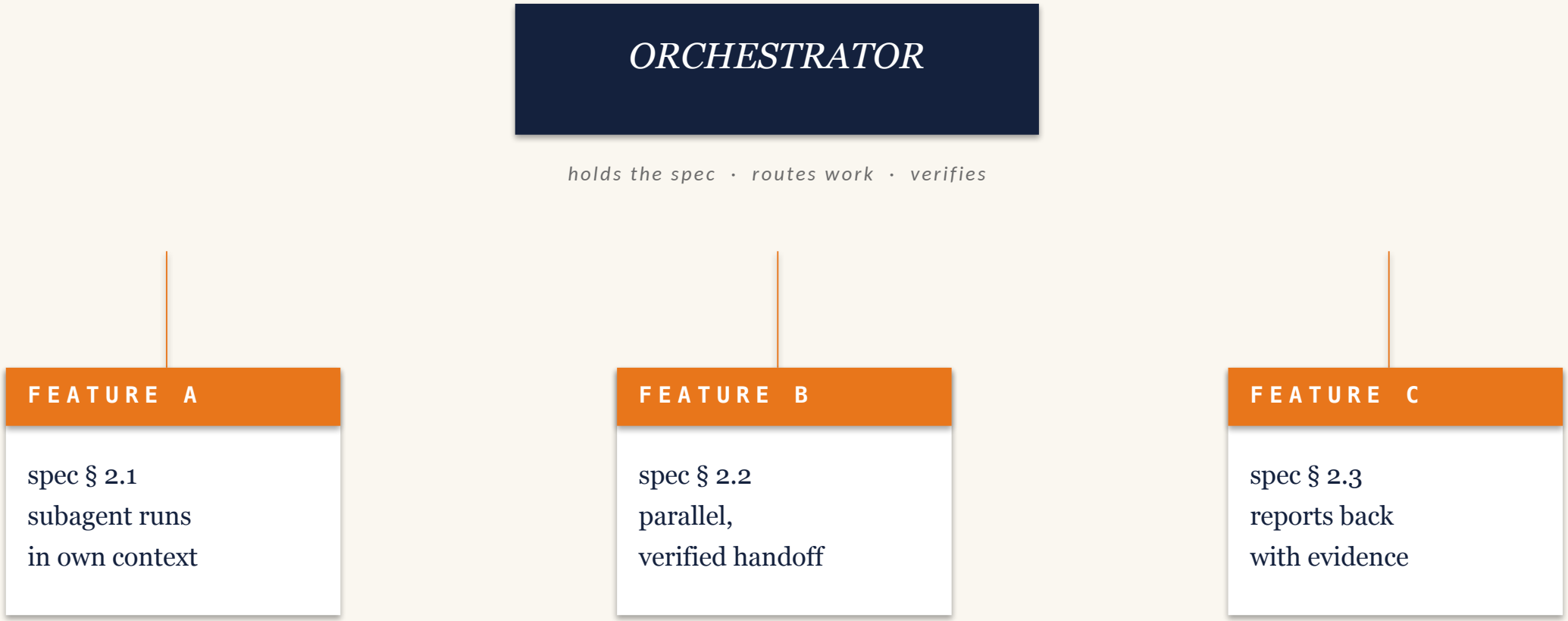
The spec stays canonical. Plans regenerate against it.

Preventing drift.

01	Spec is in version control	Reviewable like code. Every change has a PR.
02	Plans checked against the spec	Before approving a plan, ask: does it stay inside?
03	Tests reference spec line numbers	test docstring: “spec.md §2.3”. Drift visible in CI.
04	Mid-build asks update the spec first	If the spec doesn't change, the feature doesn't.
05	Subagents inherit the spec, not the chat	Pass spec as a file reference, not chat context.

One team lead. N subagents per feature. Verified handoffs.

The orchestrator pattern.



Two patterns. Pick by the shape of the work.

Orchestrator vs Conductor.

ORCHESTRATOR

One ensemble. One context.

- One agent runs the show
- Subagents return summaries
- Best for: research, refactor, planning
- Cheap and predictable
- Limit: serial bottlenecks

CONDUCTOR

Many agents. Bounded subtasks.

- Each agent owns a feature end-to-end
- Message bus / shared task list
- Best for: parallel features, async work
- More expensive, more coordination
- Limit: handoffs are where bugs live

The verify gate has to keep working when you're not looking.

Deployment.

```
# .github/workflows/ci.yml
name: verify-before-ship
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -e .[test]
      - run: pytest -q --cov
      - run: ruff check .
      - run: playwright test
      - run: ./scripts/smoke.sh staging

# main is locked. PRs only.
# smoke.sh hits a real URL on staging.
```

GATES

- Tests pass
- Linter clean
- E2E green
- Smoke test on staging
- Spec criteria checked off
- Manual approval for main
- Auto-rollback on health-fail

Spec to running service to alert to patch to redeploy. One rhythm.

The build-to-prod loop.

01

SPEC

Checkable success criteria

02

PROJECT

Repo, CLAUDE.md, plan mode

03

BUILD

TDD rhythm, verify gate, artifact

04

DOCKER

Dockerfile, compose, healthcheck

05

DEPLOY

CI/CD, smoke test, rollback

06

MAINTAIN

Logs, alerts, patch loop

Maintain feeds back into Spec. A bug ticket becomes a new line in the spec.

Same five-step rhythm at every box. Claude does the typing. You hold the gates.

Same image on your laptop and on the box. No more 'works on my machine'.

Wrap it in Docker.

```
# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY src/ ./src
HEALTHCHECK CMD curl -f localhost:8000/health
CMD ["uvicorn", "src.main:app", "--host=0.0.0.0"]

# docker-compose.yml
services:
  api:  { build: ., ports: ["8000:8000"],
          env_file: .env,
          restart: unless-stopped }
```

WHY IT MATTERS

- **Reproducibility**
  Same binary, every host.
- **Health checks**
  Restarts itself when broken.
- **Local-prod parity**
  compose up runs what CI runs.
- **One-line deploy**
  Push image. Pull image. Done.
- **Easy rollback**
  Previous tag, one command away.

Claude writes this for you. Same template every time.

Shipped is not done. Done is shipped, observed, fixable.

When prod cries, Claude reads the logs.

LOGS

Structured JSON.
Levels: debug, info, warn, error.
One line per request, with a request-id.
Write to stdout — Docker captures it.

ALERTS

Sentry / OTLP / Logflare.
Group by error fingerprint.
Page on rate spike,
not on single events.
Link alert → trace → log line.

PATCH LOOP

Alert fires. Hand Claude the trace.
Claude reads logs, reproduces locally,
adds a failing test, fixes, redeploys.
Verify gate is the same as before.

```
$ ssh prod 'docker logs api --since 10m | grep ERROR' \
    | claude -p 'find the cause, write a failing test, then fix it'
```

Logs are evidence. Evidence is what Claude is good at.

Eight items. Walk through them every time. No exceptions.

Production checklist.

- ☐ Spec criteria each ticked off
- ☐ Tests green locally AND in CI
- ☐ Feature exercised end-to-end (not just unit-tested)
- ☐ One adversarial input tried, behaved as expected
- ☐ Smoke test on staging green
- ☐ Rollback path verified (one command)
- ☐ Observability in place — logs, error alerts
- ☐ PR description references spec, plan, evidence

DISCUSSION · 05 MINUTES

Where in your work today could a spec save you the most pain?

PROMPTS

- A client request that always comes back vague
- A feature you've rebuilt twice already
- A piece of code nobody remembers the reason for

→

Beyond coding.

Claude Code is an agent harness. Code is just one surface.

It edits files. Also reads emails, opens browsers, writes notes, runs cron.

Claude Code is not a code editor.

CODE

src/, tests/, deploy

BROWSER

Chrome MCP, scrapes, forms

CLAUDE CODE

an agent harness

FILES

Obsidian, PDFs, spreadsheets

APIS

Notion, Slack, Linear, Gmail

The PM work nobody actually wants to do. Delegate it.

Claude as co-PM.

Sprint reviews	Read the PRs, write demo notes, draft retro.
Status writeups	Daily standup digest from Slack threads.
Blocker triage	Scan tickets, group by theme, propose owners.
Risk surfacing	Compare burndown to scope. Flag silently dropped items.
Decision logs	Every accepted plan auto-logged with context.

Anything you do twice a week is a candidate.

Daily automations.

```
$ crontab -l
# Vibecoding daily automation
0  6  *  *  *   ~/.bin/digest.sh          >> ~/.log/digest.log 2>&1
0  18 *  *  *   ~/.bin/standup-prep.sh    >> ~/.log/standup.log 2>&1

$ tail ~/.log/digest.log

2026-05-14 06:00  digest started
2026-05-14 06:00  scanning 27 new PDFs in ~/Drop/inbox/
2026-05-14 06:01  claude → summarise each ≤120 words
2026-05-14 06:01  claude → tag by topic, link related notes
2026-05-14 06:02  wrote 27 notes to ~/Vault/00-Inbox/digests/
2026-05-14 06:02  emailed daily digest to elias@quershift.com
2026-05-14 06:02  digest done in 2m 14s

$ head -25 ~/Vault/00-Inbox/digests/2026-05-14.md
type: digest
```

Cron + Claude + a few scripts. Your inbox, triaged. Every morning.

The web is a UI. UIs are scriptable. So are people, occasionally.

Browser automation.

› Data extraction	Scrape competitor pricing weekly. Slack a diff.
› Form filling	Submit timesheets. Pull data from project tools.
› QA replay	Walk through critical flows. Screenshot results.
› Admin work	Onboarding portals, expense filings, account cleanup.
› Research	Open 20 tabs, summarize each, output markdown brief.

Drafts, summaries, triage. You stay the final say.

Email and calendar.

```
> claude
"Summarise unread emails since Friday.
Group by sender. Flag what needs reply today.
Draft replies for me to review."

[claude] mcp__gmail__list_messages ...
[claude] mcp__gmail__read_message ... × 47

Inbox (47 unread, since 2026-05-10):
  • 3 from clients    →  reply today    [drafts ready]
  • 12 newsletters    →  auto-archive   [done]
  • 8 from team       →  read & reply   [drafts ready]
  • 24 misc           →  triaged later  [labelled]

  → Drafts saved. Open Gmail to review and send.
```

House rule: never let Claude send. Always review the draft.

Every MCP server is a verb Claude can use.

Integrations through MCP.

SERVER	VERBS YOU GET
NOTION	create-pages, fetch, search, update-page
SLACK	send-message, read-channel, search, create-canvas
GMAIL	list, read, reply (drafts only by rule)
CALENDAR	list-events, create-event, find-free-slot
LINEAR	list-issues, create-issue, update-status
GITHUB	via gh CLI — issues, PRs, releases, runs
CHROME	navigate, click, get-text, screenshot
FS	with permissions — your filesystem + your vault

Capture, distill, link. Your brain isn't a database — your vault is.

The Second Brain method.

01

CAPTURE

Anything that resonates. Lower the bar to entry.

02

DISTILL

Progressively summarise: highlight → bold → summary → gist.

03

LINK

Connect to existing notes — backlinks make the graph.

Claude Code is the perfect Distill + Link tool. Capture is still you.

A SessionEnd hook writes today's note. Tomorrow's session reads it.

Obsidian + Claude Code.

```
~/Documents/Obsidian Vault/
├── 00-Inbox/
│   └── Inbox.md
├── 01-Projects/
│   ├── Vibecoding Masterclass.md
│   ├── Inexogy Invoice Service.md
│   └── ECIS 2026 Paper.md
├── 03-Knowledge/
│   ├── Spec-Driven Development.md
│   ├── Multi-Agent Patterns.md
│   └── Claude Code Hooks.md
├── 04-People/
│   ├── Mahnoor Shah.md
│   └── Khan, Güzey, Nielsen.md
├── 05-Meetings/
│   └── 2026-05-14 SUST Triannual.md
├── 08-Sessions/
```

THE LOOP

Session starts: hook injects today's project note as context.

Session ends: hook captures what you worked on, what was decided, what's open.

No state is lost. No retelling tomorrow.

What's running on my machine right now. Yours can look similar.

Example: my own setup.

Claude Code	Primary terminal interface, two named sessions, prompt caching on
MCP servers	Notion, Slack, Gmail, Chrome, Filesystem (permissioned)
Skills	~50 personal skills — German comm, GDPR, debugging, plan-writing
Hooks	SessionEnd → Obsidian, pre-commit → tests + lint, push → vault log
Obsidian Vault	PARA structure, 800+ notes, daily session captures, linked
Cron + Routines	06:00 digest, 18:00 standup prep, weekly review on Fridays
Worktrees	Per-feature isolated branches, parallel work without churn

Most of your job isn't writing code. Most of your code isn't writing code.

Workflows for non-coders.

› Proposals & funding	Draft a Förderantrag. Match required structure.
› Status reports	Monthly client update from project tool + git activity.
› Client comms	Tone-correct drafts in your language, your style.
› Research	“Read these 15 PDFs and tell me what they disagree on.”
› Spreadsheets	Clean messy CSVs. Build pivot tables. Generate charts.
› Decks	Like this one. Generated by the script you're looking at.

WHAT YOU COULD BUILD FOR YOURSELF

One weekend.
One useful tool.

Spec it. Plan it. Test it. Ship it. By Sunday evening.

DISCUSSION · 05 MINUTES

Which integration would change your day the most?

PICK FROM YOUR LIST

- Email - drafts, triage, summaries
- Calendar - find-a-slot, prep notes
- Notion / Linear / Slack - status, blockers
- Browser - admin work, scraping, forms
- Vault - capture, distill, link

→

AI engineering with Claude Code.

Scaffold the pipeline. Read the numbers. Judge the model.

Claude doesn't invent new architectures. It wires up the boring parts.

Where Claude shines in ML.

GOOD AT

- Data prep & cleaning
- Dataloader scaffolding
- Training loop boilerplate
- Hyperparameter sweeps
- Eval metric reporting
- Result visualization
- RAG pipelines
- LoRA / adapter fine-tuning

STILL YOUR JOB

- Choosing the problem
- Designing the dataset
- Picking the right loss
- Diagnosing why metrics moved
- Reading model failures
- Deciding when it's good enough
- Ethical scrutiny
- Buying the GPU time

From "I have a CSV" to "I have a trained model with eval" — in 30 minutes.

Scaffolding a training pipeline.

```
> claude
"I have reviews.csv with `text` and `label` columns.
Build me a sentiment classifier. TF-IDF + Naive Bayes or LR.
Show me train/test accuracy and a confusion matrix."

[claude] /plan
Plan:
  1. Load CSV → DataFrame, drop NA, stratified split
  2. Pipeline: TfidfVectorizer → LinearSVC
  3. Fit, predict, report classification_report + cm
  4. Save metrics to results.json
  5. Smoke test: predict on hand-crafted pos + neg

Approve? (y/n)
```

The pipeline is mechanical. The judgement is yours.

Real dataset. Real classifier. Real test accuracy.

Demo 5 in detail.

```
Loading 20-newsgroups subset (4 topics)...
Train: 2323 docs  ·  Test: 1546 docs
Classes: graphics, hockey, space, guns

Training...

Test accuracy:  0.895

Confusion matrix (rows = actual, columns = predicted):
           graphics    hockey     space      guns
  graphics       353         3        28         5
    hockey         4       369        15        11
     space        23         7       346        18
      guns        13         2        34       315

Per-class report:
              precision    recall  f1-score   support
computer...       0.90      0.91      0.90       389
```

4 topics. 2,323 train docs. LinearSVC. 89.5% test accuracy. Fifteen minutes.

What to look at. What to ignore.

Training output, decoded.

01 Accuracy ≠ everything

A 95% accurate spam filter that misses every actual spam is useless. Look at recall on the rare class.

02 Confusion matrix > single number

Which classes does it confuse? That's where you'd add data.

03 Train >> test accuracy?

Overfitting. Regularize, simplify, or get more data.

04 Per-class precision/recall

“macro avg” weights every class equally. “weighted avg” weights by support.

An eval harness is just a test suite for your model.

Evaluation harnesses.

01	Hold out a real test set	Never train on it. Don't peek.
02	Define what good means	Numbers from the spec. Accuracy, FP rate, latency.
03	Run the harness in CI	Every model change. Same as code tests.
04	Add adversarial cases	Things you've seen fail. Edge inputs. Known-hard examples.
05	Log + compare runs	Today vs yesterday. Track regressions like bugs.

For most teams in 2026: RAG beats fine-tuning. Try it first.

Retrieval-Augmented Generation.

01	02	03	04	05
INGEST	**EMBED**	**STORE**	**RETRIEVE**	**GENERATE**
Read your docs. Split by paragraph or section.	Each chunk → vector via embedding model.	Vector DB — LanceDB, pgvector, Pinecone.	User query → vector → top-k chunks.	Stuff chunks into the prompt. Claude answers grounded.

Claude Code scaffolds all five steps. You bring the docs and the questions.

When prompt + RAG aren't enough. Rare. But real.

Fine-tuning, briefly.

› LoRA / QLoRA adapters	Train a small adapter on top of a frozen base model. Fast and cheap.
› Hugging Face skills	Claude Code has a `hf-skills-training` flow that wraps the boilerplate.
› Domain vocabulary	Best use case: highly specialized terminology a base model doesn't know.
› Style / format	Second-best: you need consistent output in a specific shape.
› Not a substitute for data quality	If your prompt-engineering doesn't work, fine-tuning rarely fixes it.

Most “we should train a model” problems aren't model problems.

When ML training is the wrong tool.

✗ When a regex or SQL query would do	Don't train a classifier to find “invoice” in emails.
✗ When the rule is known and simple	Write the rule. Tests pass. Move on.
✗ When labelling will take longer than the project	Time-box. If labelling > 1 week, reconsider.
✗ When prompting + RAG already works at 90%	Get the last 10% with rules + human review.
✗ When you can't measure success	Without an eval set, you have a vibe-classifier.

AN OPEN QUESTION

What should the AI engineer of 2027 know how to do alone?

Discuss with your buddy for two minutes.

→

The wider toolbox.

Claude Code is one of many. Know the others.

IDE-native. Fast. Parallel agent tabs.

Cursor.

✓	VS-Code-derived UI	Pleasant for IDE-first workflows
✓	Cloud agents on isolated VMs	Worktree per task, parallel
✓	Inline edits	Faster than terminal for small changes
✗	Stalls at ambiguous decisions	Without supervision
✗	No long-running autonomous mode	Not built for overnight work
✗	Less context flexibility	Tied to current file/workspace

Open-source. Model-agnostic. Bring your own provider.

Cline · Codex CLI · Aider.

CLINE

Apache 2.0

- VS Code + JetBrains + Zed
- Bring your own model
- OpenAI / Anthropic / Bedrock
- 5M+ installs
- Great for cost-sensitive teams

CODEX CLI

OpenAI

- Terminal-first, OpenAI-only
- Open-source harness
- Strong on patches & refactors
- Good plugin ecosystem

AIDER

OG, model-agnostic

- The original terminal CLI
- Auto-commits to git
- Bring your own model
- Gold standard for pair-coding

Anthropic's “supervised autonomous” mode. The future of overnight work.

Anthropic Routines.

› Package a Claude config	Prompt + repos + tools + permissions, as one bundle
› Schedule it	Nightly, hourly, or on GitHub events (PR opened, release tagged)
› Runs on Anthropic-managed cloud	Not on your laptop. Your laptop can sleep.
› Result-validation loops	Re-runs if tests fail, escalates if they keep failing
› Handoffs to humans	Pings you when judgement is needed; otherwise keeps going

Quick decision matrix. Read it once, internalise.

When to use which.

YOUR NEED	PICK	WHY
Long autonomous runs	**Claude Code + Routines**	Built for it
Inline IDE editing speed	**Cursor**	Fastest small-change loop
Cost-sensitive, open-source	**Cline**	BYO model, free harness
Terminal pair-programming	**Aider or Claude Code**	Both terminal-native
OpenAI-only stack	**Codex CLI**	Best fit for that model family
Production workflows	**Claude Code**	Plan mode, hooks, MCP, skills
Multi-agent orchestration	**Claude Code**	First-class subagents
Browser-driving tasks	**Claude Code + Chrome MCP**	Best surface today

Where this is going, late 2026 and beyond.

What's coming.

→ Multi-agent harnesses become first-class	Less DIY orchestration; more declarative agent teams.
→ Agentic IDEs	VS Code-style tools but agent-native — Cursor 4, Zed AI, Windsurf.
→ Cheaper long-running autonomy	Routines + cheap inference make overnight work the default.
→ Verified handoffs as primitive	Subagents return cryptographic “done” proofs, not just text.
→ Tighter loops with formal methods	Specs as types. Plans as checked artifacts. Less drift.

DISCUSSION · 04 MINUTES

If you had to pick one tool for your next project - which?

RULES

State the project. State the choice. Defend it in one sentence.

→

Your turn.

Three hours. The repo. Pair up. Go deep.

Everything from today, plus the demos, plus the exercises.

Clone this.

```
$ git clone \
    github.com/coronell123/
    pbb-vibecoding-masterclass.git
$ cd pbb-vibecoding-masterclass
$ bash check.sh

  ✓ Node.js 20+
  ✓ git
  ✓ Claude Code installed
  ✓ api.anthropic.com reachable

  All good. See you on the day.
```

OR SCAN

github.com/coronell123/pbb-vibecoding-masterclass

If check.sh isn't green, raise your hand. Buddy starts without you.

Ten exercises. Work in order until the break, then mix freely.

The ten exercises.

01	First contact	Read code, plan an improvement — no edits.
02	Specs	Convert a fuzzy client wish into checkable criteria.
03	Plans	Critique three plans. Reject what's wrong.
04	TDD	Red → green → refactor on validate_german_phone.
05	Debug	Name the root cause before the fix.
06	Subagents	Spawn three parallel investigators. Verify one claim.
07	Orchestrator	Feature-by-feature build with verified handoffs.
08	Playwright	Generate E2E tests for the Pomodoro app.
09	Second Brain	Wire up an Obsidian SessionEnd hook.
10	ML scaffold	Train your own classifier. Read the numbers.

Plus 11-build: pick a project, ship it, demo it.

Mixed pairs. High-experience with low. Buddy stays for the day.

Trios, not duos.

01

Pair up by show of hands

Experience: 1 (never) → 5 (daily). Highs and lows pair.

02

Negotiate before prompting

Two humans align on the prompt. Claude is the third party.

03

Driver/navigator rotates

Every 15 minutes. Both seats teach you different things.

04

Talk through your plan out loud

Your buddy is the first reviewer. Claude is the second.

On the wall the whole time. Live by them today.

Four rules for the room.

01 Trios, not duos.

You, your buddy, and Claude. Two humans negotiate before either prompts.

02 Show your plan before your code.

If you're stuck and ask me, the first question back is “what's your plan?”

03 Evidence wins.

“It works” is not done. Show the green test. Show the screenshot. Verify.

04 No real client data.

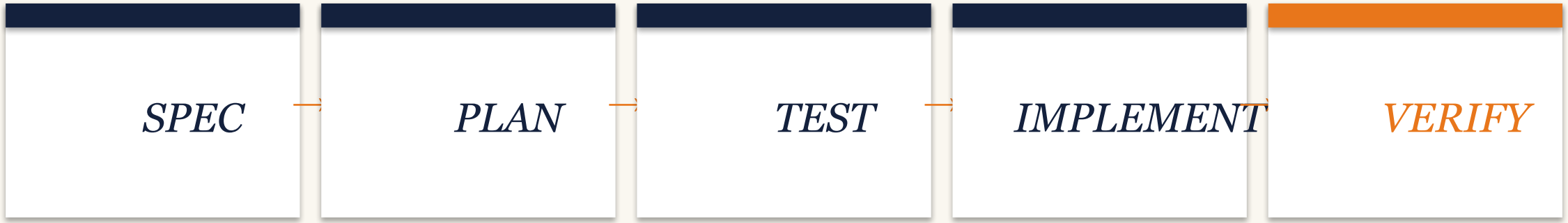
Synthetic data only. No keys, no PII, no NDA-covered material.

YOUR TURN

Go.

If you take one thing home, take this.

The rhythm, again.



In that order. Every time. Skip Verify and you didn't finish.

Before any code: five questions. Write the answers down.

Frame the problem first.

01

What do I have?

Shape of the data. Rows, columns, label balance, how clean. Open the CSV. Look at it. One sentence describing it.

02

What do I want?

Task type — classify, predict a number, cluster, generate. Be precise. A 'sentiment classifier' is not a task — '3-class sentiment on customer reviews' is.

03

What does good look like?

A number with a threshold. 'macro-F1 ≥ 0.80 on held-out 20%' is checkable. 'Works well' is not. Decide before you train, not after.

04

What's the constraint?

CPU-only. Trains in under a minute. Fits on my laptop. Runs as a small Docker container. State this up front — it rules out the wrong models.

05

What's the dumb baseline?

Most-frequent-class. TF-IDF + logistic regression. If the dumb thing already clears the bar, ship that and go home.

Five answers become one spec. Hand the spec to Claude.

The spec I'd hand to Claude.

```
# SPEC — sentiment classifier on reviews

DATA        reviews.csv  ·  8k rows  ·  text + label
TASK        3-class classification (pos/neg/neutral)

SUCCESS     macro-F1 >= 0.80 on held-out 20%
            confusion matrix to results.json

CONSTRAINTS  CPU only  ·  train < 60s  ·  Docker
              exposes /predict and /health

BASELINE    most-frequent — beat by 30 pts
MODEL       TF-IDF + LinearSVC
NON-GOAL    no fine-tuning, no transformers
```

THE RHYTHM

PLAN

Plan mode. Right model? Honest split?

TEST

Smoke test before train. 3 in, labels out.

IMPLEMENT

Approve. Claude writes pipeline + eval.

VERIFY

Run it. Beat baseline? Honest confusion?

CONTAINER

Dockerfile + compose. docker run, /predict.

DEPLOY

Push image. Smoke staging. Logs flowing.

The thinking is yours. The typing is Claude's.

We spent today on the right way. Here's the other half.

When NOT to use Claude Code.

✕ High-stakes security-critical code	Human review is non-negotiable.
✕ Code touching real client data	Confidentiality rules apply. Personal accounts only.
✕ When you don't know what you want yet	Talk to a human first. Claude can't decide what to build.
✕ Five-line tweaks	Just type them. Overhead isn't worth it.
✕ When the problem isn't a software problem	Most “tech” problems are coordination problems.

Bookmark these. Office hours open for two weeks.

Resources.

Workshop repo	`github.com/coronell123/pbb-vibecoding-masterclass`
Anthropic docs	`code.claude.com/docs/en` · the source of truth
Anthropic Skilljar (free)	Claude Code in Action — official video course
Cheatsheet	`_handouts/cheatsheet.md` — top commands and anti-patterns
Plan Critique Checklist	`_handouts/plan-critique-checklist.md` — the five questions
Office hours	30-min slots for two weeks. Email me to book.

ONE THING TO TAKE HOME

Spec.
Plan.
Test.
Implement.
Verify.

IN THAT ORDER

EVERY TIME

Thank you.